

行政事业单位内控评价合规审查智能体软件-软件说明书

行政事业单位内控评价合规审查智能体软件
软件说明书

版本号：V1.0

申请人	【请填写申请人 / 单位名称】
软件名称	行政事业单位内控评价合规审查智能体软件
版本号	V1.0
开发完成日期	2026-06-08
首次发表日期	2026-06-08
开发语言	Python 3.11 / JavaScript (ES2022) / HTML5 / CSS3
运行平台	Linux (Docker) / 现代浏览器 (Chrome / Edge / Safari)

内控评价合规审查智能体

AI 替你逐项核查被检查单位的内控评价材料——把审计师 数天的人工核查压缩到分钟级，让你专注在最需要判断的环节。

一、行业痛点

行政事业单位每年要做一次内部控制评价并上报上级。审计师和复核人面对的现实是：

- 被检查单位提交几十至上百份佐证材料（制度文件 / 会议纪要 / 财务报表 / 合同台账.....）
- 要按 54 项评价指标逐项判断每份材料是否合规、是否真实、是否对应正确指标
- 每条指标还要查法规依据、扣分理由、写专业评语
- 人工核查一个单位耗时 2-3 个工作日，重复劳动多、易遗漏、判断标准不一

二、产品定位

站在 上级复核方 视角，给审计师一把 AI 助手。

不是替代审计师，而是把重复性的”看、对、抄”工作交给 AI，让审计师专注在真正需要专业判断的环节——评分调整、修订意见、定稿签字。

三、核心价值

维度	传统方式	本系统
单个单位核查耗时	2-3 工作日	5-15 分钟
材料绑定到指标	人工分类 1-2 小时	AI 阅读 1-2 分钟 / 命中率 91-100%
出底稿	手抄 / Excel 录入	AI 自动生成 1:1 模板格式
跨单位材料查重	几乎做不到	自动比对全库材料指纹
法规依据	凭记忆翻文件	向量检索自动引用条款
标准一致性	因人而异	同一套规则跑所有任务

四、核心能力

1. AI 阅读自动绑定材料

上传整个文件夹 → AI 读取每份材料内容（不是看文件名）→ 自动识别属于 54 项指标的哪一项。

实测：23 份未绑定材料，文件名是「德市图〔2026〕1 号 ...」这种编号，AI 阅读后 23/23 全部正确绑定，命中率 100%。

2. AI 阅卷打分

对每条指标按「核查要点 + 扣分规则」给出： – 核查后得分（0 到指标满分之间） – 审计师评语初稿（可改） – 7 个判定维度：真实可靠 / 完整 / 合规 / 跨单位重复 / 匹配度

3. 工作底稿在线编辑

每条指标一行，单元格直接编辑： – 改「核查后得分」→ blur 自动保存 – 写「调整得分说明」→ 自动保存 – 点 chip 切换「材料真实可靠 / 涉嫌造假」等判定

4. 状态机与定稿

草稿（AI 生成）→ 复核中（人工编辑）→ 已定稿（锁定为只读）

定稿后底稿不可改，确保上报材料和系统数据完全一致。

5. Excel 底稿 1:1 复刻模板

底稿可一键下载为 Excel，列顺序 / 表头 / 合并单元格 / 复选框格式完全按官方核查工作底稿模板，可直接上交。

6. 跨任务材料查重

材料上传时自动计算指纹，与库内其他单位的材料对比。A 单位用 B 单位的材料”抄送”将被标记。

7. 5 大问题维度统一归类

所有 AI 检出的疑点严格归到 5 大维度： – 真实性问题（公章 / 签字 / 年度 / 正式性） – 完整性问题（要素 / 章节 / 材料缺失） – 合规性问题（制度 / 流程 / 法规违规） – 重复性问题（跨单位材料重复） – 匹配性问题（材料与指标不匹配）

8. 法规库 RAG

支持上传上位法 / 评价办法 / 编报指南等。AI 核查时自动检索相关条款，每条问题附带「依据法规第 X 条」。

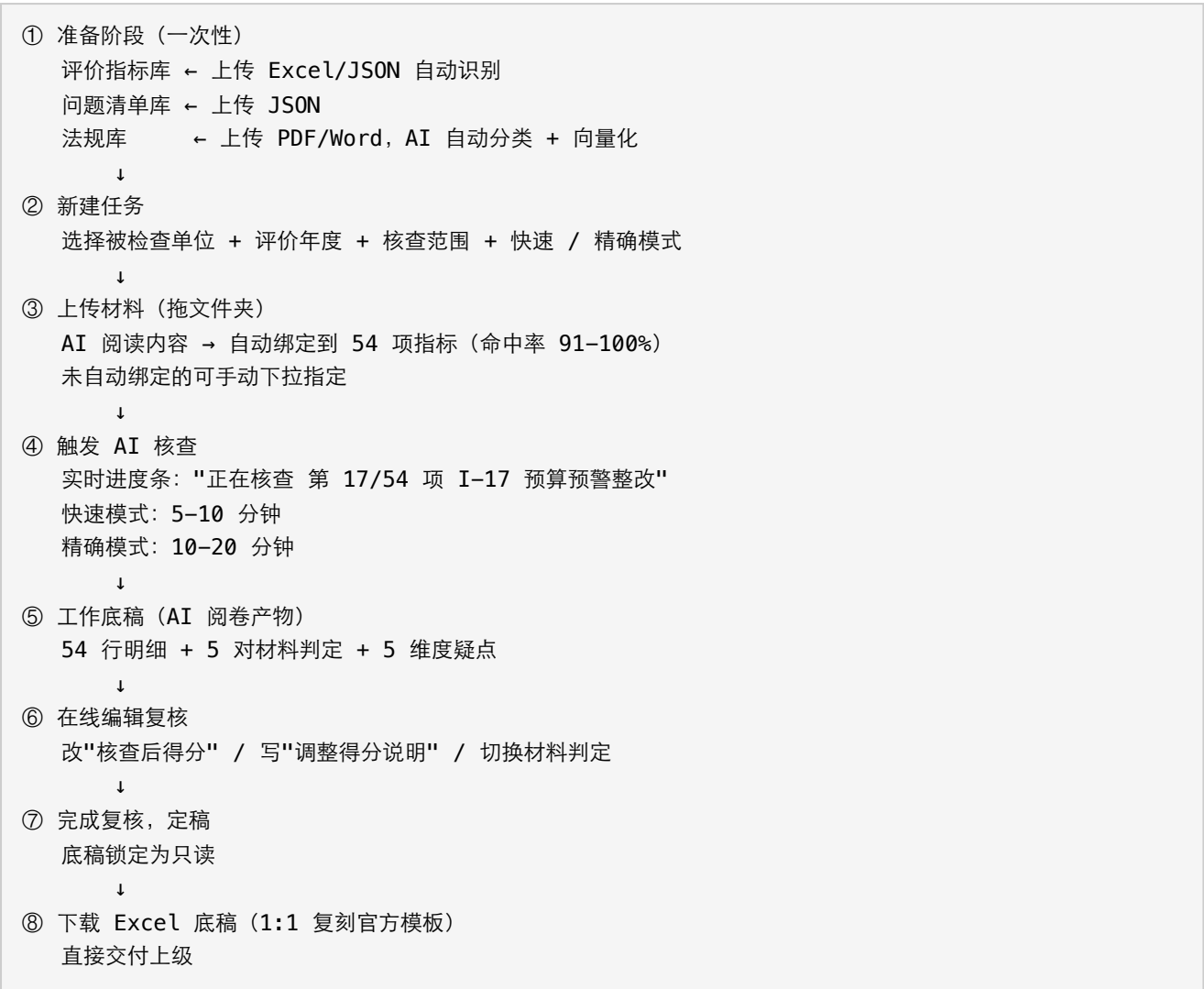
9. 全程留痕可溯源

每次”改单元格 / 忽略疑点 / 定稿”都写入审计日志。后台管理可查”谁、什么时候、改了什么”。

10. 角色权限分级

- 超级管理员：所有权限 + 解锁定稿底稿
- 审查员：核查 / 复核 / 定稿
- 被检查单位：仅能看自己单位的任务
- 只读用户：只看不改

五、典型 workflow



六、性能指标（实测）

任务规模	模式	耗时
23 份材料 / 全 54 项指标	快速模式	8-12 分钟
23 份材料 / 全 54 项指标	精确模式	15-25 分钟
5 份材料 / 选定 5 项指标	快速模式	<2 分钟
单份材料自动绑定指标	—	<3 秒

七、技术与部署

项	选型
AI 大模型	DeepSeek V4 Pro（兼容 Claude / 国产模型）
向量数据库	Qdrant

项	选型
后端	Python FastAPI + Celery
数据库	PostgreSQL
部署	Docker Compose 一键编排 7 个服务
私有化部署	✅ 全套服务可部署到客户机房，数据不出内网
离线 / 内网环境	✅ 支持 stub 模式（无 LLM 时退化为规则引擎）
国产化适配	✅ 支持 DeepSeek / 文心一言 / 通义千问
浏览器支持	Chrome / Edge / Safari 现代版本

八、安全与合规

- 所有数据存储在客户私有服务器，AI 调用可配置使用国产大模型
- 用户密码 bcrypt 加密存储
- 操作日志全留痕，可追溯任意一条数据修改的”人 / 时间 / 内容”
- 角色级数据隔离（被检查单位用户只能看到自己单位的数据）
- 支持 HTTPS + 反向代理部署

九、与传统方案对比

维度	人工 Excel 表格	通用 OA 流程	本系统
标准化程度	❌ 每人一套	⚠️ 流程标准但内容不标准	✅ 54 项指标 + 5 维度强约束
AI 自动核查	❌	❌	✅
跨任务材料查重	❌	❌	✅
底稿与报告联动	❌ 手动同步	⚠️ 流转复杂	✅ 底稿即报告
部署成本	0	高（定制开发）	低（Docker 一键起）
升级迭代	0	慢	快（开源仓库）

十、适用客户

- 各级 行政事业单位内部审计部门
- 主管部门的 内控评价复核组
- 第三方审计 / 咨询机构
- 教育 / 医疗 / 文化 等公益一类二类事业单位

十一、联系与试用

- 部署形态：私有化部署 / 公有云托管

- 试用周期：1 个被检查单位完整核查（约 30 分钟出底稿）
- 二次开发：评价指标 / 问题清单 / 法规库均可上传自定义
- 培训：1 小时操作培训 + 半天复核师傅带教

本产品采用 Apache 2.0 协议开放源代码，企业可自由部署、修改、二次开发。商业服务支持联系项目维护方。

内控评价合规审查智能体平台

完整技术开发文档 v3.0

平台定位：站在上级复核方视角，AI 拿着问题清单和法规，对被检查单位的内控评价报告与佐证材料逐项核查，出具核查报告。

核心价值：把原来需要人工数天完成的材料核查，压缩到分钟级。

开发路线：自研（不依赖 Dify）· 本地 Docker 测试 → 服务器交付

一、产品逻辑梳理

1.1 四步工作流本质

第一步：法规库	→ AI 的判断依据（上位法、评价办法、编报指南）
第二步：问题清单库	→ AI 的检查模板（真实性清单 + 常见问题清单）
第三步：检查报告+佐证材料	→ AI 的被检查对象（被审查单位提交的材料）
第四步：输出核查清单	→ AI 的成果（一单位一报告）

1.2 AI 扮演的角色

不是填报辅助工具（站在填报单位角度帮它打分）

而是上级复核审查官（站在主管部门/财政部门角度，审查填报单位材料是否真实、合规、完整）

1.3 检查维度（来自客户清单）

维度一：总体合规性 – 材料真实性（非 PS、非截图、非篡改、可追溯原件） – 材料年度一致性（必须为 2025 年度，无跨年替代） – 材料正式性（非草稿、非白条、非非正式纪要） – 材料要素完整性（公章、签字、页码、关键页齐全）

维度二：相关性核查（按 6 大业务分类） – 组织层面：三重一大、轮岗、内控会议、内控培训 – 预算业务：预算制度、公开、绩效管理 – 收支业务：票据台账、支出审批 – 政府采购：采购全流程文件 – 资产/建设/合同：盘点表、项目全流程、合同台账 – 内部监督：检查方案、整改闭环

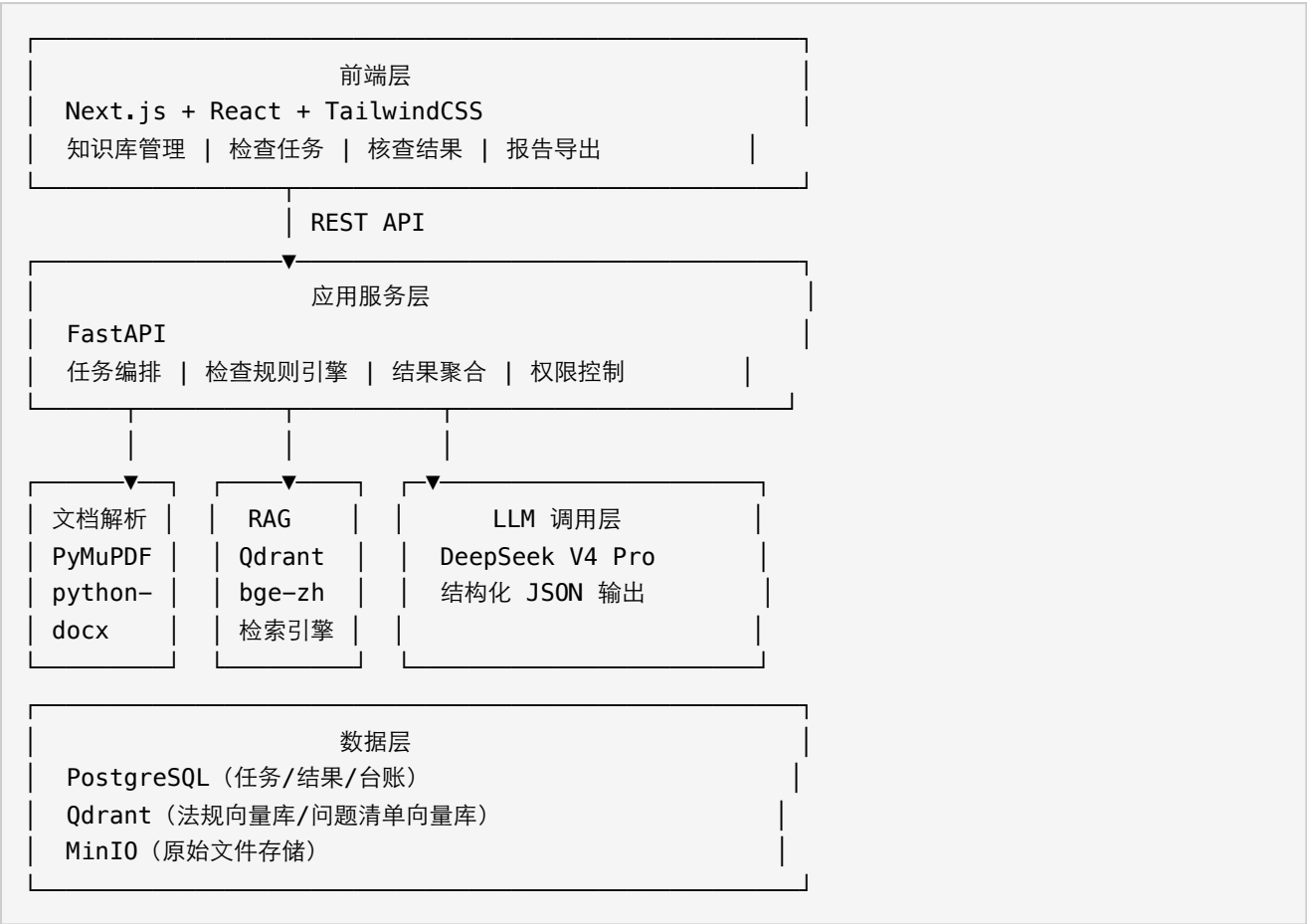
维度三：评分合规性（常见错误审查） – 关键岗位轮岗扣分是否合规 – 不适用业务处理是否正确 – 整改状态填报是否按时点判断 – 补充指标是否符合规范（总分必须 10 分）

维度四：复核规范性 – 复核权限是否越权/缺位 – 复核岗位是否分离 – 复核内容是否完整 – 复核流程是否合规

维度五：报告编报合规性 – 报告基本信息是否准确 – 内控情况描述是否真实 – 问题整改填报是否如实 – 系统操作是否规范（账号、树形结构、签章）

二、系统架构

2.1 整体架构



2.2 技术选型

组件	选型	理由
后端	Python FastAPI	异步、AI 任务友好
向量库	Qdrant	metadata 过滤、分类检索
Embedding	bge-large-zh-v1.5	中文效果强、可本地部署
LLM	DeepSeek V4 Pro	长文本、结构化输出稳定
PDF 解析	PyMuPDF	普通电子 PDF
Word 解析	python-docx	—
任务队列	Celery + Redis	检查任务耗时，必须异步
数据库	PostgreSQL	业务数据、核查结果
文件存储	MinIO	原始材料存储
前端	Next.js 14 + React	—

组件	选型	理由
报告生成	docx-js（浏览器端）	Word 核查报告导出

三、核心模块详细设计

3.1 模块一：法规知识库

内容组成：

知识库	具体内容	分块策略
上位法库	《行政事业单位内部控制规范》《政府采购法》《政府会计制度》等	按条款切分，metadata 带条款号
评价办法库	《行政事业单位内部控制评价办法》	按条款切分
编报指南库	财政部 2025 年编报指南（附件1单位指标+附件2部门指标）	按指标切分，metadata 带指标ID、扣分细则、常见扣分情形
高频问题库	客户提供的常见问题清单（全文结构化存储）	按问题类型切分

关键设计：附件1/2的常见扣分情形是 RAG 的黄金数据 每个指标的”常见扣分情形举例”要单独存储为独立 chunk，检索时精确召回，作为 AI 判断的参照标准。

更新机制：支持管理员随时上传新文件，自动触发解析→分块→向量化入库，无需重启服务。

3.2 模块二：问题清单库

内容来源： – 客户提供的佐证材料核查清单（真实性+相关性清单，按6大业务分类） – 客户提供的常见问题清单（评分问题+复核问题+报告问题） – 可持续补充历年审计发现、复核意见

存储结构：

```
{
  "check_item_id": "ZS-001",
  "category": "总体合规性",
  "subcategory": "真实性",
  "check_description": "材料是否加盖公章、签字齐全、要素完整",
  "applicable_indicators": ["1-1-1", "1-2-3"],
  "risk_level": "high",
  "common_patterns": [
    "会议纪要缺少参会人员签字",
    "制度文件无发文日期",
    "审批单签字不完整"
  ],
  "check_method": "rule", // rule(规则检查) or llm(AI判断)
```

```
"keywords": ["公章", "签字", "盖章"]
}
```

两类检查方法： - rule：确定性规则判断（如检测关键词是否存在、日期格式是否正确），不走 LLM，快且准 - llm：需要理解语义的判断（如”会议内容是否与内控相关”），走 LLM

3.3 模块三：材料接收与解析

输入范围（客户已明确）：电子文件为主

文件类型	解析工具	特殊处理
PDF（电子版）	PyMuPDF	提取文本 + 表格结构 + 页码信息
Word（.docx）	python-docx	提取正文 + 表格 + 标题层级
Excel（.xlsx）	openpyxl	提取所有 Sheet 的结构化数据
图片 PDF（扫描件）	PaddleOCR	OCR 后按页存储，标注”扫描件”

解析输出统一结构（位置信息贯穿全流程）：

```
{
  "file_id": "f-001",
  "file_name": "三重一大制度.docx",
  "file_type": "docx",
  "is_scanned": false,
  "pages": [
    {
      "page": 1,
      "section": "第一章 总则",
      "content": "...",
      "has_signature": true,
      "has_seal": true
    }
  ],
  "tables": [],
  "key_elements": {
    "has_official_seal": true,
    "has_signature": true,
    "issue_date": "2025-03-15",
    "document_number": "XXX发[2025]5号"
  }
}
```

key_elements 由规则引擎自动提取，用于真实性核查的刚性判断。

材料与指标的绑定：

用户上传材料时需指定该材料对应哪个评价指标，系统记录绑定关系。这是相关性核查的基础。

3.4 模块四：AI 核查引擎（核心）

这是整个系统最重要的模块，分为两条处理路径：

路径A：刚性规则检查（不走 LLM，快）

针对能用规则判断的问题清单条目：

```
# 示例：要素完整性检查
def check_required_elements(file_parsed, indicator):
    issues = []

    # 检查公章
    if not file_parsed["key_elements"]["has_official_seal"]:
        issues.append({
            "type": "missing_seal",
            "description": "材料未加盖公章",
            "risk_level": "high",
            "location": "全文"
        })

    # 检查发文日期
    if not file_parsed["key_elements"]["issue_date"]:
        issues.append({
            "type": "missing_date",
            "description": "材料无发文日期，无法确认是否为2025年度材料",
            "risk_level": "high"
        })

    # 检查年度一致性
    if file_parsed["key_elements"]["issue_date"]:
        year = extract_year(file_parsed["key_elements"]["issue_date"])
        if year != 2025:
            issues.append({
                "type": "wrong_year",
                "description": f"材料日期为{year}年，非2025年度材料",
                "risk_level": "high"
            })

    return issues
```

刚性规则覆盖的检查项（不走 LLM）： – 公章/签字检测 – 年度一致性（日期提取） – 文件编号格式 – 必要材料缺失（某指标规定必须有会议纪要，但用户上传的是制度文件） – 补充指标总分校验（是否等于 10 分） – 整改状态时点判断

路径B：LLM 语义检查（走 AI，深度分析）

针对需要理解内容的检查项：

Prompt 设计（结构化输出为 JSON）：

你是一名专业的行政事业单位内控评价审查专家。
请对以下材料进行核查，判断其是否满足对应评价指标的要求。

【评价指标】
{indicator_content}

【评分细则】
{deduct_rules}

【常见扣分情形】
{common_deductions} ← 来自RAG检索附件1/2

【相关法规条款】
{legal_basis} ← 来自RAG检索法规库

【被核查材料内容】
{material_content}

【核查问题清单】
{check_items} ← 来自问题清单库

请严格按照以下JSON格式输出核查结果，不得添加任何额外内容：

```
{
  "overall_judgment": "pass|warning|fail",
  "findings": [
    {
      "check_item_id": "对应的问题清单ID",
      "finding_type": "真实性问题|相关性问题|完整性问题|合规性问题",
      "severity": "high|medium|low",
      "description": "具体问题描述（客观、简洁、可引用）",
      "evidence_location": "问题在材料中的具体位置（页码/章节）",
      "legal_basis": "对应法规条款（只能引用检索到的内容，不得编造）",
      "suggestion": "整改建议"
    }
  ],
  "missing_materials": ["缺少的材料清单"],
  "irrelevant_materials": ["与指标不相关的材料清单"]
}
```

注意：

1. 只引用上方提供的法规内容，不得编造任何法规条款
2. 问题描述必须引用材料中的具体内容或位置
3. 如材料满足要求，findings 为空数组

核查任务调度（异步执行）

一个单位的材料可能有几十个文件、几十个指标，全部串行处理会很慢。

并行策略： – 同一单位的不同指标核查任务并行执行（Celery 并发） – 刚性规则检查先跑（毫秒级），LLM 检查后跑 – 前端实时显示进度（WebSocket 推送）

任务创建

↓

拆分为 N 个子任务（每个指标一个子任务）

↓

Celery 并发执行

├— 子任务1: 指标1 刚性检查 + LLM检查
├— 子任务2: 指标2 刚性检查 + LLM检查
├— ...
└— 子任务N: 指标N 刚性检查 + LLM检查

↓

所有子任务完成后聚合结果

↓

生成核查报告

3.5 模块五：核查结果管理

结果数据结构：

```
{
  "task_id": "task-001",
  "unit_name": "XX市卫生健康委员会",
  "eval_year": 2025,
  "check_date": "2026-01-15",
  "checker": "AI初核",
  "status": "completed",
  "summary": {
    "total_indicators": 45,
    "checked_indicators": 45,
    "pass_count": 28,
    "warning_count": 12,
    "fail_count": 5,
    "total_findings": 23,
    "high_risk_count": 8,
    "medium_risk_count": 10,
    "low_risk_count": 5
  },
  "findings_by_category": {
    "真实性问题": 5,
    "相关性问题": 8,
    "完整性问题": 6,
    "合规性问题": 4
  },
  "indicator_results": [...]
}
```

结果展示页面设计：

- **总览面板：** 通过/警告/不通过指标数量，高/中/低风险问题数，雷达图
- **问题列表：** 按风险等级排序，支持按类型/指标筛选，每条问题可展开查看详情
- **人工复核区：** 审查员可对每条 AI 发现标注“确认/忽略/调整”，留下复核意见

- **材料预览：** 点击问题可联动查看对应材料的具体位置（高亮显示）

3.6 模块六：核查报告生成

—单位一报告，格式：

封面

被检查单位名称

检查日期

检查依据（法规清单）

第一部分：总体核查结论

核查结论（通过/有条件通过/不通过）

问题分布统计表

核查说明

第二部分：佐证材料总体合规性核查

2.1 真实性核查结果

2.2 年度一致性核查结果

2.3 正式性核查结果

2.4 要素完整性核查结果

第三部分：各指标相关性核查明细

按组织层面/业务层面/内部监督分类

每个问题：问题描述 | 材料位置 | 法规依据 | 风险等级 | 整改建议

第四部分：高频风险点汇总

真实性高风险（PS/篡改/缺章缺字）

相关性高风险（材料与指标不匹配）

完整性高风险（关键材料缺失）

第五部分：整改建议清单

按优先级排序（高风险优先整改）

每条建议含整改要点和建议完成时限

签章页

导出格式： Word (.docx)，前端 docx-js 生成，无需后端参与。

3.7 模块七：协同复核流程

AI 初核完成（自动）

↓

人工审查员复核（标注确认/忽略/调整AI发现）

↓

生成正式核查报告（含人工复核意见）

↓

报告下发被检查单位

↓

被检查单位填写整改说明

↓
审查员确认整改闭环

权限设计：

角色	权限
超级管理员	全部权限，管理知识库、用户、任务
审查员	创建任务、查看结果、复核标注、生成报告
被检查单位	上传材料、查看本单位核查结果、填写整改说明
只读用户	查看报告，不可操作

四、开发阶段计划

阶段一：基础闭环（约 5 天）

目标：跑通单个指标的完整核查链路

- 1. 法规知识库搭建（导入编报指南附件1/2，这是最重要的数据）
- 2. 问题清单库结构化（把客户发来的清单录入数据库）
- 3. 文档解析模块（PDF + Word + Excel 解析，输出统一结构）
- 4. 刚性规则检查引擎（公章检测、年度检查、关键词检测）
- 5. 单个指标的 LLM 核查（Prompt 调试到输出稳定 JSON）
- 6. 简单的前端上传界面 + 结果展示

验收标准：上传一份”三重一大”制度文件，AI 能输出结构化核查意见。

阶段二：全量指标覆盖（约 5 天）

目标：覆盖所有评价指标，并行处理

- 1. 全量评价指标库录入（附件1单位指标 + 附件2部门指标）
- 2. 问题清单与指标的映射关系建立
- 3. Celery 并发任务调度（多指标并行核查）
- 4. 前端核查进度实时展示（WebSocket）
- 5. 核查结果聚合与展示页面

验收标准：上传一个完整单位的所有材料（10+ 文件，30+ 指标），5 分钟内完成核查并展示结果。

阶段三：报告生成与复核（约 4 天）

- 1. Word 核查报告生成（docx-js，覆盖全部章节格式）
- 2. 人工复核标注功能（确认/忽略/调整）
- 3. 整改跟踪模块
- 4. 报告下发和整改回填流程

验收标准：生成的 Word 报告格式规范，包含全部章节，问题定位可溯源。

阶段四：优化与交付（约 3 天）

- 1. 知识库管理界面（管理员可自行更新法规/问题清单）
- 2. 历史任务查询
- 3. 批量处理（同时核查多个单位）
- 4. 性能优化
- 5. 本地测试完整走通 → 部署服务器

总工期：约 17 个工作日

五、关键技术难点与对策

难点	对策
材料与指标绑定关系	上传时要求用户指定材料对应指标，不能全部丢进去让 AI 猜
扫描件 OCR 精度	客户已说明主要是电子文件，OCR 是兜底，优先处理电子版
AI 编造法规条款	Prompt 强约束 + RAG 检索结果作为唯一引用来源，输出中法规引用必须带原文片段
长文档超上下文	材料按页/章节分块，只把与当前指标相关的段落送入 LLM，控制单次 token 量
核查结果一致性	对同一材料多次核查结果可能略有差异，人工复核标注是最终确认步骤
多文件关联核查	某些问题需要跨文件比对（如制度文件和执行记录是否配套），设计跨文件关联检查任务
真实性风险（PS/篡改）	LLM 无法检测图像层面的 PS，需明确告知：AI 核查的是文本内容合规性，原件真实性需人工验原件

六、一个需要明确的边界

关于材料真实性（PS/篡改/原件可追溯）这一块，需要和客户说清楚：

- AI 能做的：检查文本内容的逻辑一致性（日期前后矛盾、数字逻辑错误、内容套模板等）
- AI 做不到的：识别图片层面的 PS 痕迹、验证是否为原件

建议在核查报告中注明：“本报告对材料内容进行逻辑核查，原件真实性需结合现场核查或档案比对确认。”

七、项目文件结构

```
audit-agent/  
├── docker-compose.yml
```

```

├── .env.example
├── frontend/
│   ├── src/app/
│   │   ├── page.tsx          # 工作台主页
│   │   ├── knowledge/       # 知识库管理
│   │   │   ├── regulations/  # 法规库
│   │   │   └── checklist/    # 问题清单库
│   │   ├── tasks/           # 检查任务管理
│   │   │   ├── create/       # 新建任务（上传材料）
│   │   │   ├── progress/     # 核查进度
│   │   │   └── [id]/         # 任务详情
│   │   ├── results/         # 核查结果
│   │   │   ├── overview/     # 总览
│   │   │   ├── findings/     # 问题列表
│   │   │   └── review/       # 人工复核
│   │   └── reports/         # 报告生成导出
│   └── src/lib/
│       ├── api.ts           # 后端 API 封装
│       ├── report-gen.ts    # Word 报告生成
│       └── websocket.ts     # 进度推送
├── backend/
│   ├── app/
│   │   ├── api/             # 路由
│   │   │   ├── knowledge.py  # 知识库 CRUD
│   │   │   ├── tasks.py      # 任务管理
│   │   │   └── results.py     # 结果查询
│   │   ├── parsers/         # 文档解析
│   │   │   ├── pdf_parser.py
│   │   │   ├── docx_parser.py
│   │   │   ├── excel_parser.py
│   │   │   └── element_extractor.py # 公章/签字/日期提取
│   │   ├── rag/             # RAG 管线
│   │   │   ├── ingest.py     # 文档入库
│   │   │   ├── chunker.py    # 分块策略
│   │   │   └── retriever.py   # 检索
│   │   ├── engine/          # 核查引擎（核心）
│   │   │   ├── rule_checker.py # 刚性规则检查
│   │   │   ├── llm_checker.py # LLM 语义检查
│   │   │   ├── orchestrator.py # 任务编排
│   │   │   └── aggregator.py  # 结果聚合
│   │   ├── llm/
│   │   │   ├── client.py     # LLM 适配层
│   │   │   └── prompts.py    # Prompt 模板
│   │   ├── tasks/           # Celery 任务
│   │   │   └── check_tasks.py
│   │   └── models/          # 数据库模型
│   └── Dockerfile
├── worker/
└── nginx/

```

八、本地 Docker 测试

```
# 1. 配置环境变量
cp .env.example .env
# 填入 ANTHROPIC_API_KEY 等

# 2. 启动全部服务
docker compose up -d --build

# 3. 初始化（首次必做）
docker compose exec backend python -m app.init_db

# 4. 导入知识库（重要：先导入附件1/2指标数据）
docker compose exec backend python -m app.rag.ingest \
  --type indicators \
  --file ./data/附件1单位评价指标.pdf

docker compose exec backend python -m app.rag.ingest \
  --type regulations \
  --file ./data/内部控制规范.pdf

# 5. 导入问题清单（结构化 JSON 格式）
docker compose exec backend python -m app.rag.ingest_checklist \
  --file ./data/问题清单.json

# 6. 访问
# 前端: http://localhost
# API文档: http://localhost:8000/docs
```

本地测试要点： – 准备一套完整的测试材料（一个单位的内控报告 + 若干佐证文件） – 故意放入一个缺章、一个跨年、一个内容不相关的材料，验证 AI 是否能识别 – 验证核查报告 Word 导出格式是否规范 – 测试并发：同时提交 3 个单位的核查任务，验证 Celery 并行处理

九、服务器部署与交付

9.1 服务器要求

配置	最低	推荐
CPU	8 核	16 核
内存	16GB	32GB
系统盘	100GB SSD	200GB SSD
数据盘	200GB	500GB
系统	Ubuntu 22.04	Ubuntu 22.04
其他	公网 IP	域名 + SSL

9.2 交付清单

- ☐ 系统部署完成，全部服务正常运行
- ☐ 法规知识库已导入（内控规范、政府采购法、编报指南附件1/2）
- ☐ 问题清单库已录入（客户提供的完整清单）
- ☐ 管理员账号创建（密码由客户自行设置）
- ☐ 完整演示一次核查流程（上传材料 → AI 核查 → 查看结果 → 导出报告）
- ☐ 演示故意放入问题材料的识别效果（关键演示点）
- ☐ 交付《操作手册》
- ☐ License 交付（收到尾款后提供完整版 Key）

十、数据安全

- 全部本地化部署，材料数据不出客户服务器
- LLM 调用只传材料文本内容，不传单位名称等敏感标识（可脱敏处理）
- 如客户要求完全数据不出境，评估使用 DeepSeek 等国产模型替代
- 账号密码由客户自行管理，开发方不接触
- 全程操作日志，可审计

— 文档结束 · v3.0 —

附录：DeepSeek V4 Pro 接入说明

模型基本信息

DeepSeek V4 Pro 是 1.6T 参数（49B 激活）的 MoE 大模型，支持 1M token 超长上下文窗口，专为复杂推理、长文档分析和 Agent workflow 设计。

API 模型字符串：deepseek-v4-pro，自 2026 年 5 月 31 日起价格永久调整为原价的 1/4，是目前全球定价最低的顶级大模型之一。

对本项目的优势： – 1M 上下文窗口：一次性处理整份内控报告+多份佐证材料，无需频繁分块 – 国内访问稳定：相比境外 API 不受网络波动影响，数据也更易满足合规要求 – 价格极低：大量调用的审计场景成本可控 – 兼容 OpenAI 格式：接入成本极低，代码几乎不用改

三种推理模式选择

DeepSeek V4 Pro 支持三种推理模式：Non-think（快速响应）、Think High（逻辑分析）、Think Max（最大推理努力）。

按本项目不同场景选择：

使用场景	推荐模式	理由
刚性规则检查（公章/日期/关键词）	不走 LLM	规则引擎直接处理，不消耗 token

使用场景	推荐模式	理由
材料相关性核查（内容是否匹配指标）	Non-think	速度快，内容理解不需要深度推理
复杂合规判断（如”制度条款是否覆盖所有环节”）	Think High	需要逻辑推导，但不必到最高档
跨文件联动核查（制度+执行记录配套分析）	Think High / Think Max	需要关联推理

API 调用代码示例

```
from openai import OpenAI # DeepSeek 兼容 OpenAI 格式

client = OpenAI(
    api_key="your-deepseek-api-key",
    base_url="https://api.deepseek.com/v1"
)

# Non-think 模式（快速，适合相关性核查）
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=[
        {"role": "system", "content": "你是内控评价合规审查专家..."},
        {"role": "user", "content": prompt}
    ],
    response_format={"type": "json_object"}, # 强制 JSON 输出
    temperature=0.1 # 审查任务要求确定性高，温度调低
)

# Think High 模式（复杂判断）
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=[...],
    extra_body={"thinking": {"type": "enabled", "budget_tokens": 8000}},
    response_format={"type": "json_object"},
    temperature=0.1
)

result = response.choices[0].message.content
```

LLM 适配层设计（支持后续切换）

在 backend/app/llm/client.py 封装为统一接口，模型通过环境变量切换：

```
# .env 配置
LLM_PROVIDER=deepseek
LLM_MODEL=deepseek-v4-pro
LLM_API_KEY=your-api-key
LLM_BASE_URL=https://api.deepseek.com/v1
```

```
# 如需切换国产备选（防止 DeepSeek 服务波动）  
# LLM_PROVIDER=qwen  
# LLM_MODEL=qwen-max
```

这样即使后续需要换模型，只改环境变量，业务代码不动。

行政事业单位文档合规协同检查智能体

自研技术开发文档 v1.0（脱离 Dify）

本文档用于指导自主开发，不依赖 Dify 等第三方 AI 平台。开发流程：本地 Docker 测试 → 部署到服务器 → 交付客户使用。

一、架构总览

完全自研，不依赖 Dify 等平台。整体分四层：

层级	技术选型	职责
前端层	Next.js + React + TailwindCSS	界面、文件上传、检查任务管理、问题台账、报告导出
应用服务层	Python FastAPI	业务逻辑、任务编排、权限控制（自研相比 Dify 多出来的主要工作量）
AI 能力层	自建 RAG + LLM 调用 + 联动校验引擎	知识检索、合规分析、跨文件交叉比对
数据层	PostgreSQL + Qdrant + 对象存储	业务数据/台账、向量知识库、原始文件

为什么自研适合这个项目：本需求的核心是「跨文件联动校验引擎」（招标↔投标↔评标↔合同的数据交叉比对），这是高度定制的多文档比对逻辑，Dify 的标准 RAG 工作流难以表达，自己写反而更直接、更可控。

二、技术选型

组件	选型	说明
后端框架	FastAPI	异步、适合 AI 任务、自动生成 API 文档
向量库	Qdrant（推荐）或 ChromaDB	Qdrant 性能好、支持 metadata 过滤（分类检索刚需）；Chroma 更轻量
Embedding	bge-large-zh-v1.5（推荐）或 OpenAI text-embedding-3	bge 中文效果好、可本地部署、免外币

组件	选型	说明
LLM	Claude 主用	中文法规理解、长文本、结构化输出稳定；通过适配层封装便于切换
PDF 解析	PyMuPDF	普通 PDF
OCR	PaddleOCR	扫描件 PDF / 图片
Word 解析	python-docx	—
Excel 解析	openpyxl	—
任务队列	Celery + Redis	批量检查、联动校验是耗时任务，必须异步
数据库	PostgreSQL	业务数据、问题台账
对象存储	MinIO（本地） / 云 OSS	原始文件存储

三、核心模块设计

3.1 文档分类与管理模块

按需求固定 9 大一级分类：

- 1. 内部制度
- 2. 合同
- 3. 采购招标（含招标 / 投标 / 评标三个子类）
- 4. 内控报告
- 5. 决算报告
- 6. 财务报告
- 7. 国有资产报告
- 8. 绩效评价报告
- 9. 其他佐证资料

每个文档存储时打上 metadata：category（一级分类）、subcategory、project_id、year、department、batch（检查批次）。这套 metadata 既用于前端二次标签检索，也用于向量检索时的过滤条件。

3.2 文档解析模块

设计成统一入口 + 策略分发：根据文件扩展名路由到对应解析器，统一输出结构化文本（带页码/章节位置信息——后续问题定位需要标注「资料具体位置」）。解析结果缓存，避免重复解析。

解析输出统一结构：

```
{
  "text": "全文文本",
  "page_blocks": [{"page": 1, "section": "第一章", "content": "..."}],
}
```



```
"tables": [],
"metadata": {}
}
```

让位置信息贯穿全流程。

3.3 RAG 知识库模块（自研重点）

脱离 Dify 后要自己搭的核心。流程：

```
法规/规则文档 → 分块(chunking) → 向量化(embedding) → 存入 Qdrant(带分类 metadata)
查询时：查询向量化 → 按 category 过滤 + 向量相似度召回 Top-K
```

分块策略针对法规文档特别处理：按「条款」切分而非固定长度，每个 chunk 保留法规名称、条款号作为 metadata，检索回来能直接引用「《政府采购法》第 X 条」作为依据。

知识库分两类： – **法规库**：上位法、财经规范（《政府采购法》《政府会计制度》《行政事业单位内部控制规范》等） – **规则库**：7 套检查模板、高频风险清单

3.4 检查规则引擎

需求区分「刚性规则」和「柔性规则」，引擎对应两条路径：

刚性规则（格式要素、流程留痕、签章完整性等）走**确定性校验**——规则配置 + 文本匹配/正则/字段提取判断，不需要 LLM，快且准。例：「发文文号是否存在」「大小写金额是否一致」。

柔性规则（条款对标上位法、权责匹配、逻辑合理性）走 **LLM + RAG**——文档内容 + 检索到的法规组装成 prompt，让 LLM 输出疑点。

预置 7 套模板（开箱即用，支持自定义增补）：

模板	适用文件
制度合规检查模板	内部制度
合同全流程检查模板	合同
招采三合一检查模板	招标/投标/评标（联动）
内控报告检查模板	内控报告
财务+决算联合检查模板	财务报告、决算报告
资产报告检查模板	国有资产报告
绩效评价报告检查模板	绩效评价报告

支持一键启用、批量套用、按单位制度增补条款。

3.5 跨文件联动校验引擎（最核心、最具差异化）

Dify 最难实现、自研最有价值的部分。设计三条联动链：

- **招采链**：招标 → 投标 → 评标 → 合同，对比标的、参数、报价、工期、服务内容一致性
- **财务链**：财务报告 → 决算报告 → 资产报告 → 合同付款，数据交叉互验
- **报告链**：内控报告 → 绩效报告 → 项目资料，内容交叉印证

实现思路（先抽取、后比对，分两阶段）：

1. 用结构化抽取（LLM 配合 JSON schema）从每份文档提取关键字段（合同金额、中标价、招标预算等），存入结构化比对表
2. 做字段级比对 + 语义比对，不一致处高亮标注

分两阶段比让 LLM 一次性读完所有文档更准、更省 token。

3.6 问题台账与报告模块

按文件类型分别生成台账（制度问题台账、招采问题台账、报告类台账）。

统计报表：完整性统计、高频违规 TOP 排行、跨文件不一致专项清单。

报告导出用 docx-js（前端生成）或 python-docx（后端生成），支持招采专项报告、综合报告、全套总报告三种。

每个问题条目统一结构：疑点描述、资料位置（文件名+页码+段落）、法规依据、问题类别、风险等级、整改建议。

3.7 协同与权限模块

角色：管理员、招采专员、财务专员、内控专员。权限按文件分类细分（财务岗只看财务类、采购岗只看招采合同类）。

工作流：

任务创建 → 资料归集 → 智能初检 → 人工协同复核（在线批注/交叉复核/逐级流转）
→ 问题下发 → 整改复核销号 → 汇总归档

全程留痕可溯源，每个操作记日志。

四、关键技术难点与对策

难点	对策
跨文件字段抽取准确率	每类文档定义明确抽取 schema，用 LLM JSON mode 强约束输出，抽取后规则校验兜底
长文档超上下文窗口	分块处理，按章节/页码分段调 LLM 后合并；或先 RAG 召回相关段落再分析
扫描件 OCR 精度	建议客户优先提供电子版；扫描件配 PaddleOCR 并人工抽检

难点	对策
AI 幻觉/误报	定位「辅助工具」，强制人工复核；prompt 约束「只引用检索到的法规，不得编造条款」
性能	刚性规则尽量不走 LLM，确定性能做的别交给模型

五、项目结构建议

compliance-agent/ ├─ docker-compose.yml ├─ .env.example ├─ frontend/ │ ├─ src/app/ │ ├─ src/components/ │ └─ src/lib/ └─ Dockerfile	# 本地测试 & 服务器部署统一编排 # 环境变量模板 # Next.js 前端 # 页面路由 # 组件 # API 封装、报告生成
├─ backend/ │ ├─ app/ │ │ ├─ api/ │ │ ├─ core/ │ │ ├─ parsers/ │ │ ├─ rag/ │ │ ├─ rules/ │ │ ├─ crosscheck/ │ │ ├─ llm/ │ │ ├─ tasks/ │ │ └─ models/ │ └─ Dockerfile	# FastAPI 后端 # 路由 # 配置、权限 # 文档解析 (pdf/docx/xlsx/ocr) # RAG 管线 (分块/嵌入/检索) # 规则引擎 (刚性+柔性+7套模板) # 跨文件联动校验引擎 # LLM 适配层 # Celery 异步任务 # 数据库模型
├─ worker/ └─ nginx/	# Celery worker (可与 backend 共用镜像) # 反向代理 + SSL

六、开发阶段建议

按以下顺序推进，先稳基础再攻难点：

阶段	目标	说明
第一阶段	单文件检查闭环	解析→刚性校验→RAG+LLM 柔性校验→问题台账→报告导出。先跑通一类文档（建议「合同」或「内部制度」，规则清晰）
第二阶段	分类扩展	扩展到 9 大分类和 7 套模板
第三阶段	联动校验引擎	先做招采链（最具代表性），再做财务链、报告链

阶段	目标	说明
第四阶段	协同与适配	协同复核、权限细分、批量处理、迎检适配

联动校验引擎最难，放在基础设施稳定后再做。

七、本地 Docker 测试流程

前期在本地用 Docker 完成开发和测试。

7.1 docker-compose 服务编排

本地一套 docker-compose.yml 拉起全部依赖：

```
services:
  frontend:      # Next.js  端口 3000
  backend:       # FastAPI  端口 8000
  worker:        # Celery worker
  postgres:      # PostgreSQL  端口 5432
  redis:         # Redis (Celery broker + 缓存)  端口 6379
  qdrant:        # 向量数据库  端口 6333
  minio:         # 对象存储  端口 9000
  nginx:         # 反向代理  端口 80
```

7.2 本地启动步骤

```
# 1. 克隆/初始化项目，复制环境变量
cp .env.example .env
# 填入 LLM API Key、数据库密码等

# 2. 构建并启动全部服务
docker compose up -d --build

# 3. 初始化数据库（首次）
docker compose exec backend python -m app.init_db

# 4. 导入法规知识库（首次）
docker compose exec backend python -m app.rag.ingest --dir ./data/regulations

# 5. 查看日志确认启动正常
docker compose logs -f backend

# 6. 访问
# 前端: http://localhost
# API 文档: http://localhost:8000/docs
```

7.3 本地测试要点

- 用真实样本文档（一份合同、一套招采资料）走完整流程，验证解析→校验→报告闭环
- 测试刚性规则的准确率（不依赖 LLM，应稳定）
- 测试 RAG 检索质量：查询法规能否召回正确条款
- 测试联动校验：故意制造招标价与合同价不一致，看能否检出
- 用 docker compose down 停止，docker compose down -v 清空数据重测

7.4 本地与生产环境差异

项	本地测试	生产服务器
对象存储	MinIO 容器	MinIO 或云 OSS
域名/SSL	localhost, 无 SSL	配置域名 + SSL 证书
资源限制	不限	按 4.2 配置资源上限
LLM	可用境外 API 测试	按数据合规要求决定境内/境外
数据	测试样本	客户真实数据

保持 docker-compose.yml 在本地和服务端尽量一致，差异通过 .env 区分。

八、服务器部署与交付

测试完成后部署到服务器交付客户。

8.1 服务器配置要求（客户提供）

配置项	最低	推荐
CPU	8 核	16 核
内存	16GB	32GB（向量库 + LLM 推理占内存）
系统盘	100GB SSD	200GB SSD
数据盘	200GB	500GB（文档 + 向量数据）
操作系统	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
其他	Docker + Docker Compose、公网 IP	域名 + SSL 证书

若 Embedding/OCR 模型本地部署且数据量大，建议配 GPU（如 NVIDIA T4），否则用 CPU 推理较慢。

8.2 部署步骤

```
# 1. 服务器安装 Docker 环境
curl -fsSL https://get.docker.com | sh
```

```
# 2. 上传项目代码 (git clone 或 scp)
git clone <repo> /opt/compliance-agent
cd /opt/compliance-agent

# 3. 配置生产环境变量
cp .env.example .env
# 填入生产数据库密码、LLM Key、域名等

# 4. 配置 Nginx 域名与 SSL (nginx/nginx.conf)
# 放置证书到 nginx/ssl/

# 5. 构建并启动
docker compose up -d --build

# 6. 初始化数据库 + 导入知识库
docker compose exec backend python -m app.init_db
docker compose exec backend python -m app.rag.ingest --dir ./data/regulations

# 7. 验证
docker compose ps          # 全部 healthy
curl https://域名/api/health
```

8.3 交付清单

- ☐ 系统部署完成，全部服务正常运行
- ☐ 法规知识库已导入，7 套检查模板已预置
- ☐ 管理员账号已创建（账号密码由客户自行设置，开发方不留存）
- ☐ 完整走通一次检查流程演示
- ☐ 交付《操作手册》《部署运维文档》
- ☐ 数据备份方案说明（数据库 + 向量库 + 文件定期备份）

8.4 运维与备份

```
# 查看服务状态
docker compose ps

# 查看日志
docker compose logs -f backend

# 数据备份（建议定时任务）
docker compose exec postgres pg_dump -U <user> <db> > backup_$(date +%F).sql
# Qdrant 数据目录 + MinIO 文件目录一并备份

# 更新部署
git pull && docker compose up -d --build
```

九、数据安全

审计数据高度敏感，遵循以下原则：

- **全部本地化部署**，数据不出境，存储在客户自己的服务器
- **LLM 调用脱敏**：如用境外 API (Claude/GPT)，只传待校验的文本片段，不传完整敏感档案；或对敏感数据部分评估国产模型 (DeepSeek、通义千问)
- **账号密码由客户自行管理**，开发方不接触、不留存
- **权限分级**：按岗位限制可见的文件分类
- **全程留痕**：操作日志可溯源

数据合规边界需在项目启动前与客户书面确认。

— 文档结束 —

内控评价合规审查智能体 · 使用手册

适用对象： 审查员 / 复核员 / 被检查单位用户 / 系统管理员 版本： V3 · 2026-06

0. 阅读导航

你的角色	重点章节
第一次用，想跑通流程	§ 1（登录） → § 4（建任务） → § 8（看底稿）
日常审查工作	§ 4-8 全套
被检查单位用户	§ 4-5（仅上传材料）
系统管理员	§ 9-11（配置 / 管理 / 维护）

1. 登录与初始化

1.1 访问地址

浏览器打开： `http://<部署服务器 IP>:8000/`

例： `http://172.16.50.100:8000/`

支持 Chrome / Edge / Safari 现代版本。不支持 IE。

1.2 首次登录

默认账号	默认密码
admin	admin123

 **登录后第一件事——立刻改密码：**

左侧导航「后台管理」 → 用户 → 找到 <code>admin</code> 行 → 点钥匙图标 → 改密码

1.3 角色说明

角色	权限
super_admin（超级管理员）	所有权限 + 解锁定稿底稿
auditor（审查员）	创建任务 / 复核 / 定稿 / 改密码
unit（被检查单位）	只能看自己单位的任务 / 上传材料

角色	权限
readonly（只读用户）	只看不改

2. 系统初始化（管理员一次性操作）

如果是全新部署的系统，按以下顺序灌入黄金数据：

2.1 评价指标库（54 项）

部署脚本会自动 seed 54 项指标。如果没有，进入：

左侧导航「评价指标」→ 右上「批量导入（JSON / PDF / Word / Excel）」
→ 上传官方核查工作底稿 .xlsx 文件
→ 系统识别表头自动解析（绿色 banner「Excel 表头自动识别」）
→ 「确认导入」

2.2 问题清单库

左侧导航「问题清单」→ 右上「批量导入」
→ 上传 check_items_seed.json

2.3 法规库

左侧导航「法规库」→ 右上「上传法规」单文件 / 「文件夹批量上传」
→ AI 自动识别文档类型（上位法 / 评价办法 / 编报指南 / 地方法规 等）
→ 自动入向量库供 RAG 检索

支持格式：PDF / Word / TXT / Markdown

2.4 配置 AI 大模型

左侧导航「后台管理」→ 大语言模型
→ 提供商选「DeepSeek」
→ 填 API Key（在 <https://platform.deepseek.com/> 申请）
→ 模型选 deepseek-chat
→ 「保存」→ 「测试连接」

不配 AI 也能跑（系统降级为规则引擎），但准确率会明显下降。

3. 创建被检查单位

每次审计前，先建立被检查单位档案：

后台管理 → 单位 → 「创建单位」

字段	说明
单位名称 *	如「XX市图书馆」
单位编码	内部编号，便于检索（选填）
级别	「单位」或「部门」

4. 创建核查任务

左侧导航「核查任务」→ 右上「新建任务」

弹窗字段

字段	说明
被检查单位 *	下拉选择已建单位（也可现场新建）
任务名 *	自由命名，如「2025年度XX单位内控评价复核」
评价年度	默认 2025
核查范围	全部指标（推荐） / 仅核查选定指标
 快速模式	勾选 = 跳过 AI 深度思考，全量约 5-10 分钟 不勾 = 精确模式，全量约 15-25 分钟

建议：首次使用 / 验证流程 → 勾选「快速模式」 + 选 5 个指标核查（2-3 分钟出结果）。

5. 上传佐证材料

5.1 进入任务详情页

任务列表点击「编号」或「任务名」→ 进入任务工作台 → 切换到「材料」标签

5.2 上传方式 A：单文件 + 绑定指标

绑定指标 ← 下拉选择该材料对应的指标（54 项之一）
材料文件 ← 选择文件
「上传」按钮 ← 点击

支持格式：.pdf .docx .xlsx .txt .md

5.3 上传方式 B（推荐）：文件夹批量上传

右上「批量上传文件夹」按钮 → 选择整个文件夹
→ 进度条显示每份文件解析情况
→ 上传完成后所有材料先归入"共享池"

5.4 AI 自动绑定指标

文件夹批量上传后，所有材料处于「未绑定」状态。点击：

「AI 自动绑定指标」按钮（材料表头）

系统会： 1. **关键词匹配**：文件名含「三重一大决策制度」等强关键词的直接绑定 2. **AI 阅读分类**：剩余未绑定的，LLM 读材料内容自动分类（30 秒-2 分钟）

实测命中率 91-100%。

完成后弹出 toast：「自动绑定 X 份（关键词 N + AI N），剩 Y 份请手动指定」。

5.5 手动绑定剩余材料

未绑定的材料行尾下拉框是**橙色高亮**，点开下拉手动选择对应指标。

下拉变更即自动保存。

5.6 全部绑定完才能触发核查

绿色横幅显示「**✓ 全部 23 份材料已绑定指标，可触发 AI 核查**」时，「触发 AI 核查」按钮变蓝可点。

若仍有未绑定，按钮为灰色 disabled 状态。

6. 触发 AI 核查

绑定完成后，点击「触发 AI 核查」按钮。

6.1 实时进度

任务工作台顶部出现进度卡片：

 AI 核查中... ⌵ 快速模式 17 / 54 项
[=====] 31%
当前指标：I-17 预算预警整改

进度每 3 秒自动刷新（不需手动 refresh）。

6.2 跑完后自动跳转到「核查发现」标签

提示「AI 初核完成，共 95 条疑点：高 22 / 中 15 / 低 58」。

6.3 防呆

已完成的任务再点「触发 AI 核查」会变成「重新核查」按钮，点击前弹 confirm 对话框：「重新核查将清空已有疑点和工作底稿，确定继续吗？」

避免误触发覆盖已有数据。

7. 复核「核查发现」(Findings)

7.1 五维度统一展示

「概览」标签底部显示 5 个维度的疑点数：

维度	说明
真实性问题	公章 / 签字 / 年度 / 正式性
完整性问题	要素 / 章节 / 材料缺失
合规性问题	制度 / 流程 / 法规违规
重复性问题	跨单位材料重复
匹配性问题	材料与指标不匹配

7.2 单条复核

「核查发现」标签 → 左侧列表点疑点 → 右侧详情显示：

- 疑点描述
- 法律依据（自动 RAG 检索）
- 整改建议

底部有 3 个按钮：

按钮	含义	对评分的影响
确认	这条疑点成立	按 100% 扣分
忽略	这条疑点无效	不扣分
调整	部分成立 / 程度减轻	按 50% 扣分

7.3 ★ 批量忽略某类疑点

「核查发现」顶部有 5 个维度按钮：

共 95 条未复核：[忽略所有 真实性问题(22)] [忽略所有 完整性问题(57)] ...

典型用法：48 条低风险的「未上传材料」提示对你来说不算”真问题”，一键忽略，剩下的才是真信号。

8. 工作底稿

8.1 进入底稿

任务工作台 → 「工作底稿」标签 → 看到 54 行表格。

8.2 列结构

#	分类	名称	核查要点	扣分规则	材料判定（5 对 chip）	标准	核查前	核查后	调整得分说明
---	----	----	------	------	----------------	----	-----	-----	--------

8.3 在线编辑

只要底稿不是「已定稿」状态，任何单元格都可编辑：

- 核查后得分：数字输入框，blur 自动保存
- 调整得分说明：textarea，blur 自动保存
- 材料判定 chip：点击切换正向/负向（5 对中各选一边）

保存状态显示在汇总行右侧（「✓ 已保存」 / 「保存中…」 / 「✗ 失败」）。

8.4 状态机

状态	含义	谁能改
草稿（AI 生成）	AI 跑完默认状态	审查员
复核中	审查员首次编辑后自动进入	审查员
已定稿	完成复核，锁定为只读	仅超级管理员可解锁

8.5 完成复核

确认底稿无误后：

工作底稿顶部「✓ 完成复核，定稿」按钮 → 弹 confirm → 确定 → 状态变为「🔒 已定稿（只读）」 → 所有单元格变为不可编辑
--

8.6 解锁（仅超级管理员）

误定稿了？切换到超级管理员账号：

工作底稿顶部「解锁底稿」按钮 → 弹 confirm → 状态退回「复核中」可继续编辑
--

9. 下载工作底稿（Excel）

工作底稿顶部「下载 Excel」按钮 → 浏览器下载「内控评价核查工作底稿_2025_{任务ID}.xlsx」
--

特点： - 1:1 复刻官方核查工作底稿模板（列顺序、表头、合并单元格、复选框） - 54 行指标 + 合计行 + 4 行签名字段 - □ / □ 字符渲染 5 对材料判定 - 可直接打印 / 上交上级

10. 常见操作

10.1 删除任务

任务详情页右上「删除任务」红色按钮 → 弹 confirm → 确定。

⚠ 会一并清除：该任务下所有材料文件、AI 检出的 Findings、工作底稿。**不可恢复。**

10.2 修改任务材料绑定

「材料」标签 → 行尾下拉重新选择指标 → 自动保存。之后**必须重新核查**（点「重新核查」按钮）才能让新绑定生效。

10.3 法规库内容预览

「法规库」→ 任意一条点「👁 查看」→ 在线阅读全文（不用下载）。

10.4 评价指标查看与下载

「评价指标」→ 任意指标点「👁 查看」→ 看核查要点、扣分规则全文。右上「导出当前列表」→ 下载 JSON。

10.5 改自己的密码

页面左下角头像旁边有一把「钥匙」图标 → 点开弹窗 → 输新密码。

10.6 全部用户管理

「后台管理 → 用户」表格：

操作	图标	说明
改密码	🔑	超级管理员可改任意用户密码
启用 / 停用	⏸	临时禁用账号
删除	🗑	软删除（用户名前加 deleted_ 前缀）

11. 系统管理（超级管理员）

11.1 后台管理菜单

左侧导航底部「后台管理」→ 4 个 tab：

Tab	内容
大语言模型	DeepSeek/Claude API Key 配置 + 思考模式
系统信息	部署版本 / 数据库类型 / 向量库状态

Tab	内容
用户	用户管理（启停 / 软删 / 改密码）
单位	被检查单位档案（创建 / 删除）
操作日志	最近 100 条操作记录（谁、何时、改了什么）

11.2 数据备份（命令行）

```
# 备份 PostgreSQL
docker compose exec -T postgres pg_dump -U compliance compliance | gzip > backup_$(date +%F).sql.gz

# 备份材料文件
sudo tar -czf storage_$(date +%F).tar.gz \
    $(docker volume inspect compliance-agent_storage_data | python3 -c "import sys,json;print(json.load(sys.stdin)[0]['Mountpoint'])")
```

11.3 升级系统

```
cd /opt/audit
git pull
cd compliance-agent
docker compose build backend worker
docker compose up -d
```

升级期间约 1-2 分钟服务不可用。

11.4 看服务状态

```
docker compose ps          # 7 个服务是否都 Up
docker compose logs -f backend worker    # 实时日志
curl http://localhost:8000/api/health    # 健康检查
```

12. 常见问题（FAQ）

- Q：AI 核查跑得很慢，怎么办？ A：默认精确模式约 15-25 分钟。建议日常用「快速模式」（创建任务时勾选 ⚡），约 5-10 分钟。或者用「选定指标」核查关键 5-10 项。
- Q：上传材料后 AI 自动绑定一个也没命中怎么办？ A：系统会优先 LLM 阅读分类（不是仅看文件名）。检查 后台管理 → 大语言模型 的 API Key 是否配置正确。没配 LLM 的话只会走关键词匹配，命中率会低。
- Q：「触发 AI 核查」按钮是灰色点不了？ A：检查「材料」标签上方的横幅是不是绿色「✓ 全部 X 份材料已绑定指标」。如果还有橙色未绑定行，需要先用「AI 自动绑定」或手动下拉指定。
- Q：跑了一半想停掉怎么办？ A：刷新页面看进度。如果真的需要停：联系管理员在服务器执行 docker compose restart worker（会强制中断当前任务）。

- Q：底稿定稿后发现要改，怎么办？ A：找超级管理员账号，进底稿点「解锁底稿」按钮。
- Q：浏览器关掉再来还能看到原来的任务吗？ A：能。所有数据存在服务器，登录回来就在。
- Q：被检查单位用户能看到其它单位的数据吗？ A：不能。系统会按 unit_id 隔离，被检查单位用户只能看到自己单位的任务。
- Q：忘记密码？ A：联系超级管理员在「后台管理 → 用户」里点钥匙图标重置。
- Q：浏览器显示「网络错误」？ A：可能服务器掉线。管理员 ssh 上服务器跑 docker compose ps 看哪个服务挂了，再 docker compose restart 服务名。

13. 操作速查卡

想做的事	路径
登录	http://IP:8000/
改自己密码	左下角钥匙图标
建任务	核查任务 → 右上新建任务
上传材料	任务工作台 → 材料标签 → 单文件 or 批量文件夹
AI 绑定	材料标签 → AI 自动绑定指标
触发核查	材料标签 → 触发 AI 核查（绿色 banner 显示后才能点）
看疑点	任务工作台 → 核查发现标签
批量忽略	核查发现 → 顶部 5 维度按钮
看底稿	任务工作台 → 工作底稿标签
改底稿	直接点单元格编辑，blur 保存
定稿	工作底稿 → 完成复核，定稿
下载 Excel	工作底稿 → 下载 Excel
删任务	任务工作台 → 右上删除任务
后台管理	左侧底部「后台管理」
改其他用户密码	后台管理 → 用户 → 钥匙图标
配 AI Key	后台管理 → 大语言模型
上传法规	法规库 → 上传法规 / 批量上传

如遇问题或有改进建议，请反馈给系统管理员。系统持续迭代中，会定期推送新版本。

内控评价智能审核系统

站在上级复核方视角，AI 拿着问题清单和法规，对被检查单位的内控评价报告与佐证材料逐项核查，出具核查报告。

详见 ../内控评价合规审查智能体-完整开发文档v3.md。

核心能力

- 法规知识库：上位法 / 评价办法 / 编报指南附件 1/2 / 高频问题
- 问题清单库：客户提供的真实性+相关性核查清单，AI 的「考题」
- AI 核查引擎：刚性规则（公章/日期/年度）+ LLM 语义核查
- 5 大检查维度：总体合规性 / 相关性 / 评分合规 / 复核规范 / 报告编报
- 协同复核流程：AI 初核 → 审查员标注 → 报告下发 → 整改销号
- 4 角色权限：超级管理员 / 审查员 / 被检查单位 / 只读用户

技术栈

层	组件
前端	单页 HTML + Vanilla JS + Tailwind CDN
后端	Python FastAPI + SQLAlchemy + Celery
LLM	DeepSeek V4 Pro（默认，1M 上下文，OpenAI 兼容） / Claude（备选） / stub（离线兜底）
RAG	Qdrant + bge-large-zh（生产） / 内存余弦（离线）
数据	PostgreSQL + MinIO + Redis

本地快速运行

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest
uvicorn app.main:app --reload      # http://localhost:8000/
```

Docker 全栈部署

```
cp .env.example .env                # 编辑端口/密码
docker compose up -d --build
```

```
docker compose exec backend python -m app.init_db  
# 默认登录 admin / admin123 (首次部署后请尽快修改)
```

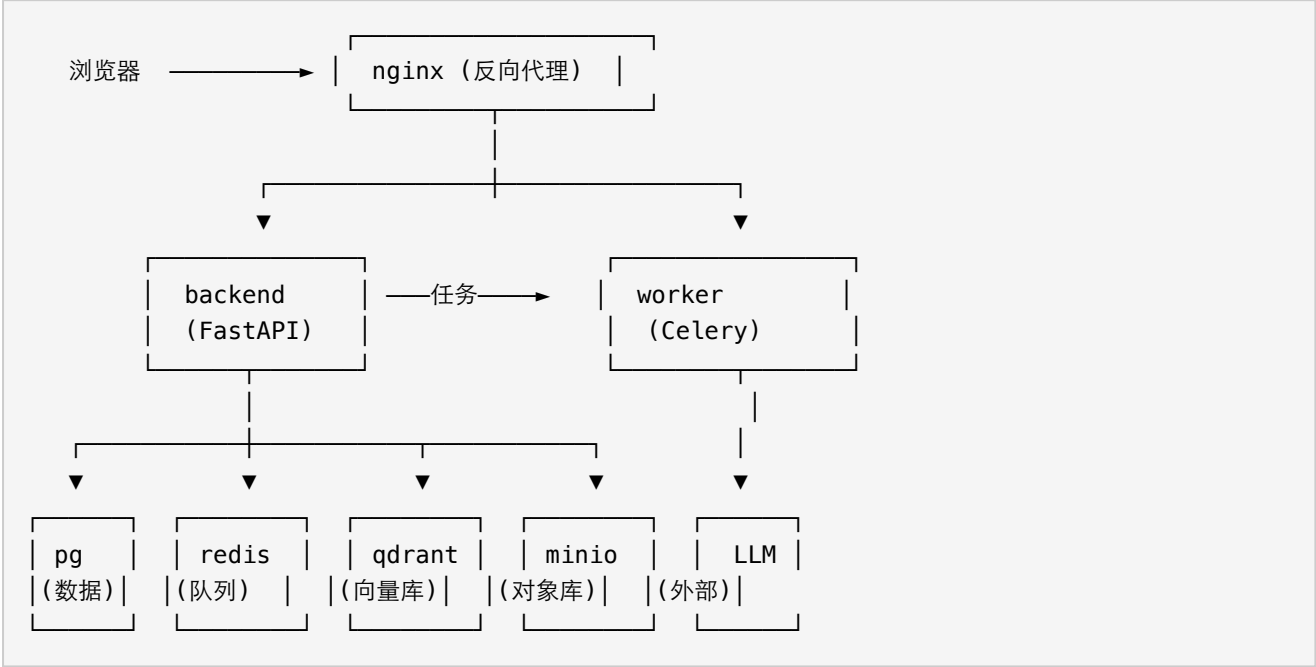
LLM API Key 配置

两种方式（任一即可）： 1. **环境变量**：.env 设置 LLM_API_KEY=sk-... 2. **后台界面**：登录后进「系统设置」页填入 DeepSeek API Key（覆盖环境变量）

附录 C：系统部署与架构

C.1 部署架构

系统采用 Docker Compose 多容器编排，七个核心容器协同提供服务：



C.2 部署步骤

1. 在 Linux 服务器安装 Docker 24+ 与 Docker Compose v2
2. 克隆代码：`git clone <仓库地址> && cd compliance-agent`
3. 复制并修改配置：`cp .env.example .env`（设置数据库密码、LLM API key 等）
4. 启动全部服务：`docker compose up -d`
5. 等待健康检查通过后访问 `http://<服务器 IP>:18080/`

C.3 数据卷与持久化

容器	数据卷	用途
postgres	pgdata	业务数据持久化
redis	redisdata	缓存与任务队列
qdrant	qdrantdata	向量索引
minio	miniodata	上传材料原文件
backend / worker	./storage	临时解析结果

C.4 备份与恢复

- 数据库：使用 `pg_dump` 定期导出
- 对象存储：MinIO Mirror 或定期 `rclone`

- 向量库: Qdrant 内置快照

C.5 监控

后端提供 `/api/health` 健康检查接口，可接入 Prometheus / 阿里云监控。

附录 A：数据库表结构说明

本系统采用 PostgreSQL 关系数据库，核心实体定义在 `backend/app/models/entities.py`，使用 SQLAlchemy 2.x 声明式映射。下列章节按模块罗列各业务实体的表名、字段含义与关联关系。

A.1 实体 User

- 表名：users
- 说明：用户 + 角色 (v3 §3.7)：super_admin | auditor | unit | readonly。
- 字段清单 (共 8 列)：
 - id
 - username
 - password_hash
 - role
 - full_name
 - unit_id
 - is_active
 - created_at

A.2 实体 AuthToken

- 表名：auth_tokens
- 字段清单 (共 4 列)：
 - id
 - user_id
 - token
 - created_at

A.3 实体 AuditLog

- 表名：audit_logs
- 说明：操作审计日志 (v3 §3.7 「全程留痕可溯源」)。
- 字段清单 (共 8 列)：
 - id
 - user_id
 - username
 - action
 - target_type
 - target_id
 - detail
 - created_at

A.4 实体 AppSetting

- 表名：app_settings

- **说明：**系统全局配置 key/value 存储（如 LLM API Key）。敏感配置如 API Key 也存在于这里（数据库级隔离即可，避免环境变量层级硬编码）。
- **字段清单**（共 3 列）：
 - key
 - value
 - updated_at

A.5 实体 Indicator

- **表名：**indicators
- **说明：**评价指标（编报指南 附件1/附件2）。例：1-1-1「三重一大」决策制度建立与执行情况，4 分。
- **字段清单**（共 13 列）：
 - id
 - indicator_code
 - level
 - category
 - subcategory
 - name
 - description
 - max_score
 - audit_points
 - deduct_rules
 - common_deductions
 - required_materials
 - created_at

A.6 实体 Regulation

- **表名：**regulations
- **说明：**法规库条目（v3 §3.1）。支持上传原始文件（PDF/Word/TXT），并按文档类型与地区分类：
 - 上位法 / 评价办法 / 编报指南 / 地方法规 / 部门规章 / 其它 – 国家 / 省 / 市 / 区县 / 部门 上传后自动解析全文，分块入 Qdrant 向量库供 RAG 检索。
- **字段清单**（共 18 列）：
 - id
 - title
 - doc_type
 - region
 - issuer
 - doc_number
 - effective_date
 - description
 - tags
 - file_name
 - storage_path
 - file_size

- file_type
- parsed_text
- chunks_count
- indexed
- uploaded_by
- created_at

A.7 实体 CheckItem

- **表名:** check_items
- **说明:** 问题清单条目 (v3 §3.2)。 例: ZS-001 真实性 – 材料是否加盖公章、签字齐全、要素完整
- **字段清单** (共 12 列):
 - id
 - item_code
 - dimension
 - subcategory
 - description
 - applicable_indicators
 - risk_level
 - common_patterns
 - check_method
 - keywords
 - is_active
 - created_at

A.8 实体 AuditUnit

- **表名:** audit_units
- **说明:** 被检查单位 (v3 §3.7 一个独立角色范畴)。
- **字段清单** (共 7 列):
 - id
 - name
 - code
 - level
 - description
 - created_at
 - tasks

A.9 实体 AuditTask

- **表名:** audit_tasks
- **说明:** 一个单位一次核查任务 (v3 §3.5)。 一个任务对应一个被检查单位的内控评价报告核查, 包含多个材料和多个指标。 支持两种核查范围 (scope): – all: 对评价指标库内全部已入库指标核查 (推荐用于全面体检) – selected: 仅对 selected_indicator_ids 列出的指标核查
- **字段清单** (共 20 列):
 - id

- unit_id
- name
- eval_year
- scope
- selected_indicator_ids
- status
- summary
- stats
- progress_current
- progress_total
- progress_text
- fast_mode
- created_by
- created_at
- completed_at
- unit
- materials
- findings
- worksheet

A.10 实体 Material

- 表名: materials
- 说明: 被检查单位上传的佐证材料 (v3 §3.3)。每份材料**必须**绑定一个评价指标 (v3 §3.3 明确要求)。
- 字段清单 (共 14 列):
 - id
 - task_id
 - indicator_id
 - file_name
 - storage_path
 - file_type
 - is_scanned
 - key_elements
 - parsed_text
 - content_hash
 - content_fingerprint
 - created_at
 - task
 - indicator

A.11 实体 Finding

- 表名: findings
- 说明: AI 核查发现 (v3 §3.4)。finding 是 v3 的核心输出对象, 每条都对应一个具体的合规疑点, 支持人工复核标注 (确认/忽略/调整) 和整改闭环。

- **字段清单** (共 24 列):
 - id
 - task_id
 - material_id
 - indicator_id
 - check_item_id
 - finding_type
 - severity
 - description
 - evidence_location
 - legal_basis
 - suggestion
 - source
 - review_status
 - reviewer_id
 - review_note
 - reviewed_at
 - rectification_status
 - rectification_note
 - rectified_at
 - created_at
 - updated_at
 - task
 - material
 - indicator

A.12 实体 Worksheet

- **表名:** worksheets
- **说明:** 工作底稿头 (与任务 1:1)。 阅卷流程: AI 跑完 → 自动建底稿草案 → (V2) 人工复核改单元格 → 定稿 → 报告。 每条指标对应一行 WorksheetRow, 本表存底稿级元数据: 被核查单位名称、组织机构代码、核查/复核签名、定稿状态。
- **字段清单** (共 11 列):
 - id
 - task_id
 - unit_name
 - unit_code
 - auditor_name
 - reviewer_name
 - status
 - created_at
 - updated_at
 - task
 - rows

A.13 实体 WorksheetRow

- **表名:** worksheet_rows
- **说明:** 工作底稿行（每条指标在一份底稿里一行，AI 打分粒度）。
- **字段清单**（共 14 列）：
 - id
 - worksheet_id
 - indicator_id
 - serial
 - original_score
 - audited_score
 - audit_finding_text
 - adjustment_note
 - material_flags
 - linked_finding_ids
 - created_at
 - updated_at
 - worksheet
 - indicator

附录 B：HTTP 接口清单

系统对外提供 RESTful HTTP 接口，前端单页应用与第三方系统通过这些接口完成所有数据交互。下表按模块罗列已发布的接口路径、HTTP 方法及业务用途简述。

B.audit 模块

HTTP	路径	处理函数
DELETE	/unit_id	delete_unit()
GET	/task_id	get_task_detail()
POST	/task_id/materials	upload_material()
PATCH	/task_id/materials/material_id	bind_material_indicator()
POST	/task_id/materials/auto-bind	auto_bind_materials()
POST	/task_id/run	run_task()
POST	/task_id/finalize	finalize_task()
DELETE	/task_id	delete_task()
GET	/task_id/report	download_task_report()
GET	/task_id/material-review	get_material_review()
POST	/task_id/materials/merge-duplicates	merge_duplicate_materials()
GET	/task_id/worksheet	get_task_worksheet()
POST	/task_id/worksheet/rebuild	rebuild_task_worksheet()
PATCH	/task_id/worksheet/rows/row_id	patch_worksheet_row()
POST	/task_id/worksheet/finalize	finalize_worksheet()
POST	/task_id/worksheet/unlock	unlock_worksheet()
GET	/task_id/worksheet.xlsx	download_task_worksheet_excel()
GET	/finding_id	get_finding()
POST	/finding_id/review	review_finding()
POST	/finding_id/rectify	submit_rectification()
POST	/finding_id/resolve	resolve_rectification()
GET	/material_id/preview	preview_material()

B.auth 模块

HTTP	路径	处理函数
POST	/login	login()
POST	/logout	logout()
GET	/me	me()
PATCH	/ {user_id} /password	change_password()
PATCH	/ {user_id} /activate	set_user_active()
DELETE	/ {user_id}	delete_user()

B.knowledge 模块

HTTP	路径	处理函数
PUT	/ {indicator_id}	update_indicator()
DELETE	/ {indicator_id}	delete_indicator()
POST	/import-from-file	import_indicators_from_file()
POST	/import	import_indicators()
PUT	/ {item_id}	update_check_item()
DELETE	/ {item_id}	delete_check_item()
POST	/import-from-file	import_check_items_from_file()
POST	/import	import_check_items()

B.regulation 模块

HTTP	路径	处理函数
GET	/ {reg_id}	get_regulation()
GET	/ {reg_id} /download	download_regulation()
GET	/ {reg_id} /content	get_regulation_content()
DELETE	/ {reg_id}	delete_regulation()
POST	/classify	classify_regulation_file()

B.settings 模块

HTTP	路径	处理函数
GET	/llm	get_llm()
PUT	/llm	update_llm()

HTTP	路径	处理函数
POST	/llm/test	test_llm_connection()

附录 D：后端核心服务模块清单

系统后端按职责拆分为多个服务模块，每个模块对应一个独立的 Python 文件，集中管理一类业务规则。下表罗列各模块的文件路径、入口类/函数及业务职责说明。

D.ai_material_classifier

- 文件：backend/app/services/ai_material_classifier.py
- 代码行数：120 行
- 模块说明：AI 阅读材料 → 自动分类到评价指标。

批量将未绑定材料发给 LLM，让它根据文件名 + 解析文本内容，决定材料属于 哪个指标。LLM 一次性返回 {material_id: indicator_code} 映射。

设计要点： – 分批处理（默认 15 份/批），避免单次 prompt 超 32K token – 每份材料只截取前 800 字（够 LLM 判断主题） – stub 模式（无真实 LLM）直接返回空，让 fallback 用关键词 – LLM 返回的 indicator_code 不在 55 项内 → 忽略 – 1 份材料最多绑 1 个指标 – 顶层符号（共 4 个）： – _format_indicator_list – _format_materials – _build_prompt – ai_classify_materials

D.audit_service

- 文件：backend/app/services/audit_service.py
- 代码行数：334 行
- 模块说明：核查任务服务（v3 §3.4、§3.5、§3.7）。

主要职责： – 创建被检查单位 – 创建任务、上传材料（绑定指标 + 自动抽取 key_elements） – 触发 AI 核查（异步入队） – 复核标注（确认/忽略/调整） – 整改闭环（提交整改 → 销号） – 顶层符号（共 9 个）： – create_unit – create_task – upload_material – auto_bind_materials – review_finding – submit_rectification – resolve_rectification – delete_task – finalize_task

D.extract_service

- 文件：backend/app/services/extract_service.py
- 代码行数：462 行
- 模块说明：（无模块说明）
- 顶层符号（共 9 个）：
 - _parse_to_text
 - _heuristic_indicators
 - _heuristic_check_items
 - extract_indicators
 - _normalize_header
 - _excel_by_header_names
 - classify_regulation
 - _classify_heuristic
 - extract_check_items

D.material_matcher

- 文件: backend/app/services/material_matcher.py
- 代码行数: 190 行
- 模块说明: (无模块说明)
- 顶层符号 (共 4 个):
 - _normalize
 - match_subcategory
 - match_indicator
 - filter_materials_by_subcategory

D.material_review_service

- 文件: backend/app/services/material_review_service.py
- 代码行数: 334 行
- 模块说明: (无模块说明)
- 顶层符号 (共 6 个):
 - detect_duplicates
 - collect_content_review
 - matching_overview
 - build_timeline
 - review_overview
 - merge_duplicate_group

D.regulation_service

- 文件: backend/app/services/regulation_service.py
- 代码行数: 152 行
- 模块说明: 法规库服务: 文件上传 / 解析 / 索引 / 删除。

文件上传后: – 存到对象存储目录 – 用 parsers 解析全文 – 按条款 chunk → Qdrant (v3 §3.3 黄金数据) – 顶层符号 (共 3 个): – ingest_regulation – delete_regulation – get_regulation_file

D.report_service

- 文件: backend/app/services/report_service.py
- 代码行数: 561 行
- 模块说明: 核查报告生成 (v3 §3.6)。

按 v3 文档规定的 5 章节结构生成 Word 报告: 封面 第一部分: 总体核查结论 第二部分: 佐证材料总体合规性核查 第三部分: 各指标相关性核查明细 第四部分: 高频风险点汇总 第五部分: 整改建议清单 – 顶层符号 (共 9 个): – build_report_docx – _build_conclusion – _count_indicators – _add_finding_table – _add_finding_detailed – _bullet – _truncate – _fmt_date – _suggested_deadline

D.scoring_service

- 文件: backend/app/services/scoring_service.py
- 代码行数: 151 行
- 模块说明: 任务评分汇总服务。

扣分数 (先用固定值, 未来支持管理员可配置): - 高风险 finding: 扣指标满分 $\times$ 50% - 中风险 finding: 扣指标满分 $\times$ 25% - 低风险 finding: 扣指标满分 $\times$ 10%

复核状态权重: - pending (未复核): 100% - confirmed (已确认): 100% - ignored (已忽略): 0% (不扣分) - adjusted (已调整): 50%

等级阈值 (百分比): - 优 $\geq$ 90 - 良 $\geq$ 80 - 中 $\geq$ 60 - 差 $<$ 60 - 顶层符号 (共 2 个): - _grade - compute_task_scoring

D.settings_service

- 文件: backend/app/services/settings_service.py
- 代码行数: 74 行
- 模块说明: 系统设置服务: 管理 LLM API Key 等运行时可改配置。

DB 中的 AppSetting 优先级高于环境变量。 管理员在后台填入 DeepSeek API Key 后立即生效, 无需重启。 - 顶层符号 (共 4 个): - _get - _set - get_llm_config - update_llm_config

D.user_service

- 文件: backend/app/services/user_service.py
- 代码行数: 131 行
- 模块说明: 用户管理服务 (密码修改 / 启停 / 软删)。

安全规则: - 不允许操作自己的启停 / 删除 - 不允许停用 / 删除最后一个 super_admin - 停用 / 删除会立即清空该用户的 AuthToken - 顶层符号 (共 3 个): - update_password - set_active - soft_delete_user

D.worksheet_export

- 文件: backend/app/services/worksheet_export.py
- 代码行数: 319 行
- 模块说明: (无模块说明)
- 顶层符号 (共 6 个):
 - _decide_issue_count
 - _collect_top_issues
 - _format_signature_year_docno
 - _format_flag_cell
 - _set_row
 - build_worksheet_xlsx

D.worksheet_service

- 文件: backend/app/services/worksheet_service.py
- 代码行数: 351 行
- 模块说明: (无模块说明)
- 顶层符号 (共 6 个):
 - _extract_self_scores

- `_material_dup_check`
- `_material_match_ratio`
- `_build_audit_text`
- `build_worksheet_draft`
- `get_worksheet`

D.engine 推理引擎模块

llm_checker

- 文件: `backend/app/engine/llm_checker.py` · 165 行
- 说明: LLM 语义检查器 (v3 §3.4 路径 B)。

按 v3 §3.4 的 Prompt 模板组装: 评价指标 + 评分细则 + 常见扣分情形 + 法规 + 问题清单 + 材料正文, 要求模型严格输出指定 JSON。

orchestrator

- 文件: `backend/app/engine/orchestrator.py` · 453 行
- 说明: 核查任务编排 (v3 §3.4)。

`run_audit`: 对一个 `AuditTask` 内所有材料执行完整核查流程 (先刚性规则、后 LLM 语义、聚合 Finding 写入 DB)。

rule_checker

- 文件: `backend/app/engine/rule_checker.py` · 280 行
- 说明: 刚性规则检查器 (v3 §3.4 路径 A)。

不调 LLM, 仅根据 `key_elements` + 关键词匹配判断。覆盖: 真实性 / 年度一致性 / 正式性 / 要素完整性 / 关键词命中 / 补充指标总分。

附录 E：版本开发历程（Git 提交记录）

本系统采用 Git 进行版本管理，自项目初始化以来累计的核心提交记录如下（按时间倒序）：

提交日期	提交摘要
2026-06-08	feature: 工作底稿 – 第 10 列改为”签章年度文号” + 新增”发现主要问题汇总”行
2026-06-08	feature: 核查发现里”材料出处”可点击 — 打开原文件预览/下载
2026-06-04	fix: 时间显示差 8 小时 — fmtTime 补 ‘Z’ 后缀按 UTC 解析
2026-06-04	ui: 全站清理 emoji 表情 — 与 SVG icon 风格统一
2026-06-04	feature: 「材料审核」新视图 — 文本查重+内容审核+附件匹配+更新情况
2026-06-04	fix: 法规库下载 401 — 改为 fetch+blob 方式（带 Authorization header）
2026-06-04	perf: AI 核查改为并发模式 — 预计提速 4–8 倍
2026-06-04	fix: 年度规则改造 — 制度类材料不强制当年印发
2026-06-04	docs: 加产品介绍 + 使用手册两份用户面向文档
2026-06-04	deploy: 加 deploy.sh 一键部署脚本 + Dockerfile 支持 pip 国内源
2026-06-04	ui: 修复上传佐证材料表单按钮与左侧列未对齐
2026-06-04	ui: 暂时隐藏「导出 Word 报告」按钮（用户当前不需要）
2026-06-04	docs: 加 README.md + .gitignore；忽略 Claude Code 本地权限配置
2026-06-04	fix: 控制疑点爆炸 — 噪音减 80%（237 → ~50） + 5 维度严格归一
2026-06-04	fix: 指标库分值对称化修正 — 模板原 98 分 → 标准 100 分
2026-06-03	fix: 3 个体验问题 — I-55 残留 / 单位删除 / 方法字段中文化
2026-06-03	ui: 评价指标库导入预览改良 — 显示核查要点/扣分规则 + 识别来源徽章
2026-06-03	V3 新版底稿模板：列重排 + 5 对复选框 + 调整得分说明 + Excel 表头智能识别

提交日期	提交摘要
2026-06-03	V2 工作底稿在线编辑 + 定稿状态机 + 报告读底稿
2026-06-03	fix+ui: 防止任务重复运行 + 维度分布合并为 5 类
2026-06-03	feature: AI 阅读材料自动分类 — 自动绑定命中率从 0% 升到 91-100%
2026-06-03	feature: 材料-指标完整绑定 UI (行内下拉 + AI 自动绑定 + 触发核查防呆)
2026-06-03	perf: 智能材料匹配 — 共享池 LLM 调用从 Nx55 降到 Nx子类指标数
2026-06-03	feature: AI 核查实时进度跟踪 + 快速模式开关
2026-06-03	hotfix: 任务删除 500 — Worksheet ↔ AuditTask 加 ORM 级联
2026-06-03	V1 工作底稿: AI 阅卷产物 (55 项指标 → Excel 1:1 复刻模板)
2026-06-03	UI 打磨: emoji→SVG 图标系统 + 登录页文案精简
2026-06-03	功能 3: 核查报告评分汇总表 (v3 §1.3 维度三「评分合规性」)
2026-06-03	功能 2: 法规库内容预览 (在线看法规原文)
2026-06-03	功能 1: 用户管理 — 改密码 / 启停 / 软删
2026-06-03	任务删除 + 评价指标/问题清单 查看与下载
2026-06-03	任务核查范围 + 材料文件夹批量上传 + 报告 5 段式格式
2026-06-02	法规库: 文件夹批量上传 + 3 种智能分类策略
2026-06-02	法规库上传 accept 列表加上 .xlsx + 增加格式说明文字
2026-06-02	评价指标 / 问题清单 支持 PDF / Word / Excel 智能导入 (含预览)
2026-06-02	修复: LLM 配置保存按钮无反馈
2026-06-02	RAG 真接通 + Word 报告 5 章节导出 (v3 §3.4 §3.6 黄金交付)
2026-06-02	新增法规库 + 评价指标/问题清单 增强: 上传/查看/搜索/删除
2026-06-02	前端: 路由级权限守卫 + 侧栏按角色显隐每个入口

提交日期	提交摘要
2026-06-02	v3 重构 · 苹果系统级 UI + 左右分屏登录 + 后台管理控制台
2026-06-02	v3 重构 · UI 重设计 + 动线优化 (檔编辑式权威风格)
2026-06-02	v3 重构 4/4: 前端重写 (v3 工作流 UI + LLM Key 设置页)
2026-06-02	v3 重构 3/4: key_elements 抽取 + 5 大维度规则引擎 + 核查服务
2026-06-02	v3 重构 2/4: LLM 适配层 (DeepSeek) + 后台 LLM Key 设置 + 知识库 API
2026-06-02	v3 重构 1/4: 清理 v1 + 重命名为「内控评价智能审核系统」
2026-06-02	fix: 添加 SVG favicon, 消除浏览器控制台 /favicon.ico 404
2026-06-02	fix: docker-compose 端口映射参数化 + 挂载 frontend 目录
2026-06-02	Phase 4: 批量上传 + 批次管理 (最后一块)
2026-06-02	Phase 4: Celery 异步任务
2026-06-01	Phase 4: 协同复核 + 在线批注 (§3.7)
2026-06-01	Phase 4: 权限分级 + 用户体系 + 审计日志 (§3.7)
2026-06-01	Phase 4 前端: 单页 UI 接入全部后端能力
2026-06-01	Phase 3 收尾: 财务链 + 报告链联动校验
2026-06-01	Phase 2 收尾: 补全剩余 4 套模板 (内控/财务+决算/资产/绩效)
2026-06-01	Phase 3: 招采链联动校验引擎
2026-06-01	Phase 2: 招采三合一检查模板
2026-06-01	Phase 2: 内部制度合规检查模板
2026-06-01	Phase 1: 单文件合规检查闭环 (合同)

附录 F：术语与缩略语

术语	含义
内控评价	行政事业单位每年按统一指标对自身内部控制体系进行自评的合规活动
评价指标	内控评价的最小检查单位，本系统内置 54 项官方指标
佐证材料	被检查单位为证明指标达成而提交的制度文件、会议纪要、报表等
工作底稿	审计师/复核员对每项指标核查结论的留痕文档（本系统在线编辑）
核查发现	系统识别出的疑似不合规、材料瑕疵或扣分项
RAG	Retrieval-Augmented Generation，检索增强生成，本系统用于法规条款查询
LLM	Large Language Model，本系统对接 DeepSeek / Claude 等大模型
向量库	存储文档语义向量的数据库，本系统使用 Qdrant
状态机	工作底稿的生命周期：草稿 → 复核中 → 已定稿
真实可靠	5 维度判定之一：材料非 PS、非截图、非篡改、可追溯原件
跨单位重复	多个被检查单位提交了内容相同/高度相似的材料
签章年度文号	材料正式性核心要素：发文机关签章 + 年份 + 文号编号
知识库	系统内置的法规、问题清单、指标说明等结构化数据集合
API	Application Programming Interface，应用程序编程接口
Token	1) HTTP 鉴权令牌；2) LLM 输入输出的最小语义单元
Embedding	嵌入向量，文本经模型编码后得到的语义向量表示
Chunk	文档切片，长文档按段落或语义边界切分后供向量检索使用
Celery	Python 分布式任务队列，本系统用于异步处理材料解析
FastAPI	Python 现代 Web 框架，本系统后端基于此构建

术语	含义
SQLAlchemy	Python ORM 库，本系统用于关系数据库映射
Docker Compose	容器编排工具，本系统使用其编排 7 个服务容器